Practical 2: Installing CRD with Docker

Day 2 — Duration: 3 Hours

SDI Training for ICT Professionals — Climate Risk Database

Based on official GeoNode documentation: `https://docs.geonode.org/en/master/install/basic/`

What you will accomplish: By the end of this practical you will have a fully working Climate Risk Database (CRD) running on your own laptop. The CRD is built on GeoNode, an open-source geospatial content management system. You will be able to open it in your browser, log in as administrator, and see the platform ready for data.

Before you start: Docker must be installed and running. Check for the whale icon in your system tray (Windows) or menu bar (Mac). If you skipped Practical 1, go back and complete it first. You also need Git and Python 3.10+ installed.

Part 1: Install System Prerequisites (Ubuntu 22.04)

Note: If you are using Windows or macOS with Docker Desktop already installed, skip to Part 2. The commands below are for Ubuntu 22.04 servers.

Step 1.1: Install required system packages:

```
sudo add-apt-repository universe
sudo apt-get update -y
sudo apt-get install -y git-core git-buildpackage \
  debhelper devscripts python3.10-dev \
  python3.10-venv virtualenvwrapper
sudo apt-get install -y apt-transport-https \
  ca-certificates curl lsb-release gnupg \
  gnupg-agent software-properties-common vim
```

What this does: Installs Git, Python development tools, and system utilities needed by GeoNode.

Step 1.2: Install Docker Engine (if not already installed):

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL \
  https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list \
  > /dev/null
sudo apt-get update -y
sudo apt-get install -y docker-ce docker-ce-cli \
  containerd.io docker-compose
sudo apt autoremove --purge
```

What this does: Adds the official Docker repository and installs Docker Engine and Docker Compose.

Step 1.3: Add your user to the Docker group:

```
sudo usermod -aG docker ${USER}
su ${USER}
```

What this does: Allows you to run Docker commands without `sudo`. You need to log out and back in (or run `su ${USER}`) for this to take effect.

Part 2: Create the GeoNode Project

Open your terminal:

- **Windows:** Open **PowerShell** (search in Start menu)
- **Mac:** Open **Terminal** (Applications → Utilities → Terminal)
- **Ubuntu:** Press `Ctrl+Alt+T`

How GeoNode installation works: The GeoNode project repository is a **Django project template** — it contains placeholder variables like `{{project_name}}` that must be rendered into real values. You first create a Python virtual environment, install Django, then use `django-admin startproject` to generate your actual project from the template. Only then can you build with Docker.

Step 2.1: Create a Python virtual environment:

```
cd ~
python3.10 -m venv ~/.venvs/my_crd
source ~/.venvs/my_crd/bin/activate
```

What this does: Creates an isolated Python environment so the packages we install do not affect your system Python. The `source` command activates it — you should see `(my_crd)` at the beginning of your terminal prompt.

If `python3.10` is not found:

- **Ubuntu:** Run `sudo apt install -y python3.10 python3.10-venv`
- **Mac:** Try `python3 -m venv` instead (macOS often has Python 3.12+)
- **Windows:** Download Python 3.10+ from `https://python.org/downloads/`

Step 2.2: Install Django (needed to generate the project):

```
pip install Django==4.2.9
```

What this does: Installs Django 4.2.9, which is needed to render the GeoNode project template into an actual project.

Step 2.3: Create the project directory:

```
mkdir ~/my_crd
```

Step 2.4: Generate the project from the GeoNode template:

```
GN_VERSION=5.0.1
django-admin startproject \
  --template=https://github.com/GeoNode/geonode-project/archive/refs/heads/
      $GN_VERSION.zip \
  -e py,sh,md,rst,json,yml,ini,env,sample,properties \
```

```
  -n monitoring-cron -n Dockerfile \
  my_crd ~/my_crd
```

What this does: Downloads the GeoNode project template (version 5.0.1) and renders it into a real project called `my_crd`. All placeholder variables like `{{project_name}}` are replaced with `my_crd`. This creates a ready-to-build Docker project.

You should see: Django downloading and extracting the template. It may take a moment.

If the above command fails, you can use the alternative local method:

```
cd ~
git clone https://github.com/GeoNode/geonode-project.git
cd geonode-project
git checkout 5.0.1
django-admin startproject \
  --template=./geonode-project \
  -e py,sh,md,rst,json,yml,ini,env,sample,properties \
  -n monitoring-cron -n Dockerfile \
  my_crd ~/my_crd
```

If you see "git: command not found":

- **Windows:** Download and install Git from `https://git-scm.com/download/win`
- **Mac:** Type `xcode-select -install` in the terminal
- **Ubuntu:** Run `sudo apt install -y git`

Step 2.5: Enter the project folder:

```
cd ~/my_crd
```

Step 2.6: Generate the environment configuration file:

The project includes a helper script called `create-envfile.py` that generates a valid `.env` configuration file.

For local development/training (simplest option):

```
python create-envfile.py
```

What this does: Creates a `.env` file with sensible defaults for running GeoNode on `localhost`. This is the recommended approach for training.

Advanced: For a production deployment with HTTPS, you can pass options:

```
python create-envfile.py \
  --https \
  --env_type prod \
  --hostname my-crd.example.org \
  --email admin@example.org \
  --geonodepwd MySecurePassword \
  --geoserverpwd GeoServerPass \
  --pgpwd PostgresPass \
  --dbpwd DatabasePass \
```

```
  --geodbpwd GeoDBPass
```

Step 2.7: (Optional) Review and customise the configuration:

```
nano .env
```

What this does: Opens the configuration file so you can review or change settings.

Key settings to check:

```
SITEURL=http://localhost/
POSTGRES_PASSWORD=your_password_here
GEOSERVER_ADMIN_PASSWORD=geoserver
SECRET_KEY=auto-generated-random-key
```

Save and exit nano: Press `Ctrl+O`, then `Enter`, then `Ctrl+X`.

Why these settings matter:

- **SITEURL:** Tells GeoNode its address. For local use, `localhost` is correct.
- **POSTGRES_PASSWORD:** Password for the database. The script generates a random default.
- **GEOSERVER_ADMIN_PASSWORD:** Password for the GeoServer map server admin panel.
- **SECRET_KEY:** Used for security (encrypting cookies/sessions). Must be unique per installation.

Part 3: Build and Start the CRD

Step 3.1: Build the Docker containers:

```
docker compose build
```

What this does: Reads the project configuration and downloads/builds all software containers — the web server, database, map server, and more.

You should see: Many lines of output as Docker downloads and builds components:

```
[+] Building 12/12
 => [django] ...
 => [celery] ...
 => downloading packages ...
```

This step takes 10–20 minutes depending on your internet speed. Docker is downloading several gigabytes of software. Do not close the terminal. If you see progress indicators, it is working.

Step 3.2: Start the CRD platform:

```
docker compose up -d
```

What this does: Starts all CRD services in the background. The `-d` flag means "detached" — containers run in the background.

You should see:

```
[+] Running 8/8
 - Container my-crd-db4geo-1 Started
 - Container my-crd-rabbitmq-1 Started
 - Container my-crd-geoserver-1 Started
 - Container my-crd-django-1 Started
 - Container my-crd-celery-1 Started
 - Container my-crd-nginx-1 Started
```

What are all these containers?

- **db4geo:** PostgreSQL/PostGIS database — stores all your spatial data
- **django:** The GeoNode web application — the website you interact with
- **geoserver:** The map server — serves map images and spatial data via OGC standards
- **nginx:** The web server — handles incoming web requests and serves static files
- **celery:** The task worker — processes background jobs (thumbnails, metadata)
- **rabbitmq:** The message broker — coordinates tasks between services

Step 3.3: Watch the startup logs:

```
docker compose logs -f django
```

What this does: Shows live log output from the Django container. Wait until you see:

```
[UWSGI] Uwsgi running...
```

This means the application is ready. This typically takes 3–5 minutes.

To stop watching logs: Press `Ctrl+C`. The containers keep running.

If you see errors: Some initial errors are normal as services wait for each other. Wait 3–5 minutes and check again.

Step 3.4: Verify all containers are running:

```
docker compose ps
```

You should see: A table showing all services with state "running" or "Up".

Part 4: Access and Verify Your CRD

Step 4.1: Open your CRD in the browser:

Open your web browser (Chrome or Firefox) and go to:

```
http://localhost
```

You should see: The GeoNode homepage with a map, search bar, and navigation menus.

Step 4.2: Log in as administrator:

The default credentials (generated by `create-envfile.py`) are:

- **GeoNode:** username: `admin`, password: `admin`
- **GeoServer** (`http://localhost/geoserver/`): username: `admin`, password: `geoserver`
- Click the **"Sign in"** button (top-right corner)
- Enter: username `admin`, password `admin`
- Click **"Sign in"**

You should see: Your username in the top-right corner and an **"Admin"** link in the menu.

> **Important:** If you set custom passwords using `create-envfile.py` flags (`-geonodepwd`, `-geoserverpwd`), use those passwords instead.

Step 4.3: Verify GeoServer is running:

In your browser, go to:

```
http://localhost/geoserver/
```

You should see: The GeoServer web administration interface.

Step 4.4: Test the REST API:

In your browser, go to:

```
http://localhost/api/v2/datasets/
```

You should see: A JSON response (may be empty if no datasets uploaded yet).

Useful Docker Commands Reference

Keep this page as a quick reference. You do not need to run these now.

1. **Check which containers are running:**

```
docker compose ps
```

2. **View logs for a specific service:**

```
docker compose logs -f django # GeoNode app
docker compose logs -f geoserver # Map server
docker compose logs -f db # Database
docker compose logs -f geonode # NGINX web server
```

3. **Stop the CRD (data is preserved):**

```
docker compose down
```

4. **Start the CRD again:**

```
docker compose up -d
```

5. **Restart a single service:**

```
docker compose restart django
```

6. **Check error logs:**

```
tail -F -n 300 invoke.log
```

Troubleshooting

Problem: Port 80 is already in use

Symptom: Error "bind: address already in use" or "port 80 is already allocated".

Solution:

```
# Find what is using port 80:
# Mac/Linux:
sudo lsof -i :80
# Windows PowerShell:
netstat -ano | findstr :80
```

Stop the conflicting program, or change the CRD port in `.env`:

```
HTTP_PORT=8080
```

Then restart: `docker compose down && docker compose up -d`

Now access CRD at `http://localhost:8080`.

Problem: Not enough memory

Symptom: Containers crash or show memory errors.

Solution:

- Open Docker Desktop → Settings → Resources
- Set Memory to at least **4 GB** (6 GB recommended)
- Click **Apply & Restart**
- Run `docker compose up -d` again

Problem: Docker is not running

Symptom: "Cannot connect to the Docker daemon" or "docker: command not found".

Solution:

- **Windows/Mac:** Open Docker Desktop. Wait for the whale icon to stop animating.
- **Ubuntu:** Start Docker:

```
sudo systemctl start docker
```

Problem: 502 Bad Gateway

Symptom: Browser shows "502 Bad Gateway" error.

Cause: Django has not finished starting yet.

Solution: Wait 3–5 minutes, then refresh. Check Django logs:

```
docker compose logs django --tail 20
```

Look for "[UWSGI] Uwsgi running..." — this means Django is ready.

Checkpoint

Before proceeding to Practical 3, confirm:

- ☐ **CRD loads:** `http://localhost` shows the GeoNode homepage
- ☐ **Admin login works:** You can sign in with admin/admin (or your custom credentials)
- ☐ **GeoServer accessible:** `http://localhost/geoserver/` shows the GeoServer interface
- ☐ **API responds:** `http://localhost/api/v2/datasets/` returns JSON
- ☐ **All containers running:** `docker compose ps` shows all services as "running"

Well done! You now have a fully functional CRD platform running on your laptop. In the next practical, you will customise it with your institution's name and branding.

End of Practical 2 — Proceed to Practical 3: Customising Your CRD